

INTERNSHIP TASK

Multi-Tenant SaaS – AI Agent Builder with RAG

Objective

Build a FastAPI-based multi-tenant SaaS platform that lets any organisation create its own AI agent in minutes. A tenant should be able to upload a document and paste a website URL, and the platform must automatically ingest both sources, build a knowledge base, generate a tailored system prompt using an LLM, and return a ready-to-use agent that answers questions grounded in that content (RAG). Each tenant's data, agents and conversations must be strictly isolated from every other tenant.

Your Task

Develop a REST API with the following endpoints:

1. **POST /tenants** — Create a tenant (organisation name, admin email) and return a unique API key. All other endpoints (except /health) must require this key in the **X-API-Key** header.
2. **POST /agents** — Create an agent with a name, an uploaded document (PDF, DOCX or TXT) and a website URL. The platform must extract text from the document, scrape the website, chunk and embed the content, and use the LLM to generate a system prompt (role, tone, scope and guardrails) for the agent.
3. **GET /agents** — List all agents belonging to the calling tenant only.
4. **GET /agents/{id}** — Retrieve an agent's details, including its generated system prompt, sources, number of indexed chunks and ingestion status (processing / ready / failed).
5. **POST /agents/{id}/chat** — Send a user message and return the agent's answer, generated from the most relevant retrieved chunks, along with the sources used (include a session ID to keep chat history).
6. **GET /health** — A simple health check endpoint.

Technology Requirements

- **Framework:** FastAPI (Python)
- **LLM:** Any LLM of your choice for system prompt generation and answering (e.g. Claude, GPT, Gemini, or an open-source model such as Llama or Mistral via Ollama)
- **Embeddings:** Any embedding model (e.g. OpenAI text-embedding-3-small, Voyage AI, or sentence-transformers)
- **Vector Store:** ChromaDB, Qdrant, FAISS or PostgreSQL + pgvector, with data partitioned per tenant
- **Database:** SQLite or PostgreSQL to store tenants, agents and chat history
- **Ingestion:** Any parsing / scraping library (e.g. PyMuPDF, python-docx, BeautifulSoup, trafilatura)
- **File Storage:** Save uploaded documents to the local filesystem, organised by tenant

Deliverables

- Working FastAPI application
- Demo showing two tenants, each with its own agent, where one tenant cannot access the other's agent or data
- README file with setup and run instructions, chosen LLM / models and a short architecture overview
- .env.example file listing required environment variables (no real API keys committed)

Notes

You are free to choose the LLM, embedding model, vector store and chunking strategy that best suit your approach. Focus on clean code, strict tenant isolation, proper error handling (invalid files, unreachable URLs, missing API keys) and a working demo. Answers should stay grounded in the uploaded content and the agent should say so when it does not know. Swagger docs should be accessible at /docs by default (FastAPI handles this automatically). A simple frontend is optional but a plus.

Good luck!